docs.nvidia.com

A Scalable and Efficient Post-Training Library — NeMo-RL

22–28 minutes

CICD NeMo RL passing

📣News#

- [10/10/2025] **DAPO Algorithm Support**
  NeMo RL now supports Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO) algorithm. DAPO extends GRPO with **Clip-Higher**, **Dynamic Sampling**, **Token-Level Policy Gradient Loss**, and **Overlong Reward Shaping** for more stable and efficient RL training. See the DAPO guide for more details.
- [9/30/2025]Accelerated RL on GCP with NeMo RL!
- [9/27/2025] FP8 Quantization in NeMo RL
- [9/25/2025] On-policy Distillation
- Student generates on-policy sequences and aligns logits to a larger teacher via KL, achieving near-larger-model quality at lower cost than RL. See On-policy Distillation.

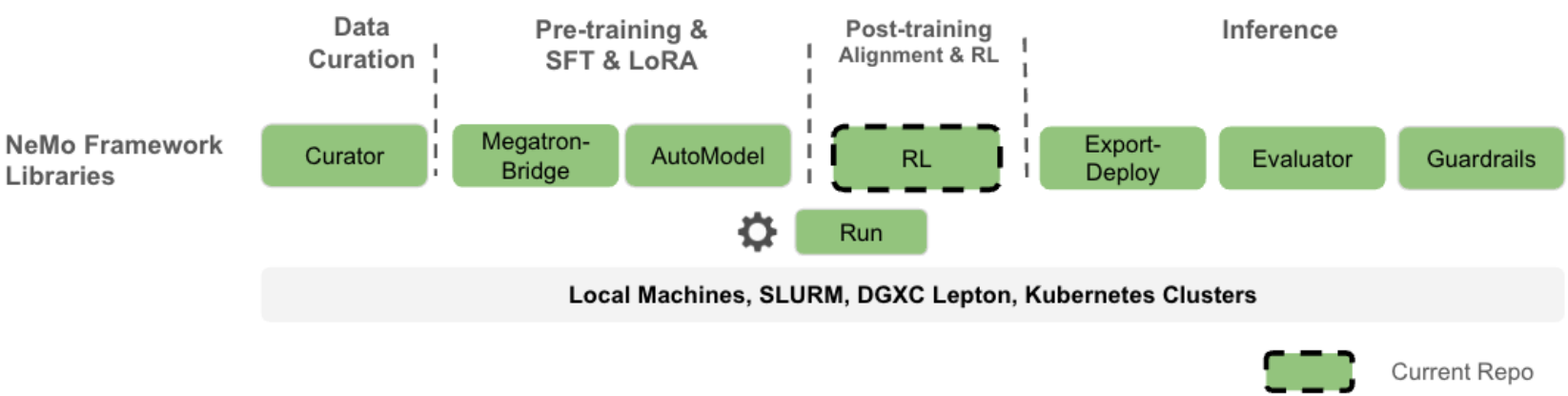
▸ Previous News

- [8/15/2025] NeMo-RL: Journey of Optimizing Weight Transfer in Large MoE Models by 10x

- [7/31/2025] NeMo-RL V0.3: Scalable and Performant Post-training with Nemo-RL via Megatron-Core
- [7/25/2025] Release v0.3.0!
- 📝v0.3.0 Announcement
- 📊View the release run metrics on Google Colab to get a head start on your experimentation.
- [5/14/2025] Reproduce DeepscaleR with NeMo RL!
- [5/14/2025] Release v0.2.1!
- 📊View the release run metrics on Google Colab to get a head start on your experimentation.

Overview#

NeMo RL is an open-source post-training library under the NVIDIA NeMo Framework, designed to streamline and scale reinforcement learning methods for multimodal models (LLMs, VLMs etc.). Designed for flexibility, reproducibility, and scale, NeMo RL enables both small-scale experiments and massive multi-GPU, multi-node deployments for fast experimentation in research and production environments.



What you can expect:

- **Flexibility** with a modular design that allows easy integration and customization.
- **Efficient resource management using Ray**, enabling scalable and flexible deployment across different hardware configurations.
- **Hackable** with native PyTorch-only paths for quick research prototypes.
- **High performance with Megatron Core**, supporting various parallelism techniques for large models and large context lengths.
- **Seamless integration with Hugging Face** for ease of use, allowing users to leverage a wide range of pre-trained models and tools.
- **Comprehensive documentation** that is both detailed and user-friendly, with practical examples.

Please refer to our design documents for more details on the architecture and design philosophy.

Training Backends#

NeMo RL supports multiple training backends to accommodate different model sizes and hardware configurations:

- **DTensor** - PyTorch's next-generation distributed training with improved memory efficiency (PyTorch-native TP, SP, PP, CP, and FSDP2).
- **Megatron** - NVIDIA's high-performance training framework for scaling to large models with 6D parallelisms.

The training backend is automatically determined based on your YAML configuration settings. For detailed information on backend selection, configuration, and examples, see the Training Backends

documentation.

Generation Backends#

NeMo RL supports multiple generation/rollout backends to accommodate different model sizes and hardware configurations:

- **vLLM** - A high-throughput and memory-efficient popular inference and serving engine.
- **Megatron** - A high-performance Megatron-native inference backend which eliminates weight conversion between training and inference.

For detailed information on backend selection, configuration, and examples, see the Generation Backends documentation.

Features#

✅ *Available now* | ➡️ SOON *Coming in v0.4*

- ➡️ SOON **Megatron Inference** - Megatron Inference for fast Day-0 support for new Megatron models (avoid weight conversion).
- ➡️ SOON **Async RL** - Support for asynchronous rollouts and replay buffers for off-policy training, and enable a fully asynchronous GPRO.
- ➡️ SOON **Vision Language Models (VLM)** - Support SFT and GRPO on VLMs through the DTensor path.
- ➡️ SOON **Improved Native Performance** - Improve training time for native PyTorch models.
- ➡️ SOON **Improved Large MoE Performance** - Improve Megatron Core training performance and generation performance.

- ➡️SOON **End-to-End FP8 Low-Precision Training** - Support for Megatron Core FP8 training and FP8 vLLM generation.
- ➡️SOON **Megatron Bridge Integration** - Integrate Megatron Bridge to enable training features from Megatron Core.
- ➡️SOON **NeMo Automodel Integration** - Integrate NeMo Automodel to power our DTensor path.
- ➡️SOON **New Models** - gpt-oss.
- ➡️SOON **Expand Algorithms** - DAPO, GSPO.
- ➡️SOON **GB200** - Add container support for GB200.
- ✅ **Distributed Training** - Ray-based infrastructure.
- ✅ **Environment Support and Isolation** - Support for multi-environment training and dependency isolation between components.
- ✅ **Worker Isolation** - Process isolation between RL Actors (no worries about global state).
- ✅ **Learning Algorithms** - GRPO/GSPO, SFT, DPO, and On-policy distillation.
- ✅ **Multi-Turn RL** - Multi-turn generation and training for RL with tool use, games, etc.
- ✅ **Advanced Parallelism with DTensor** - PyTorch FSDP2, TP, CP, and SP for efficient training.
- ✅ **Larger Model Support with Longer Sequences** - Performant parallelisms with Megatron Core (TP/PP/CP/SP/EP/FSDP).
- ✅ **MoE Models** - Support for DeepSeekV3 and Qwen-3 MoE models (Megatron).
- ✅ **Sequence Packing** - Sequence packing in both DTensor and Megatron Core for huge training performance gains.

- ✅**Fast Generation** - vLLM backend for optimized inference.
- ✅**Hugging Face Integration** - Works with 1B to 70B models (Qwen, Llama).

Table of Contents#

- Prerequisites
- Quick Start
- Support Matrix

Algorithms	Single Node	Multi-node
GRPO	GRPO Single Node	GRPO Multi-node: GRPO Qwen2.5-32B, GRPO Multi-Turn
On-policy Distillation	Distillation Single Node	Distillation Multi-node
Supervised Fine-Tuning (SFT)	SFT Single Node	SFT Multi-node
DPO	DPO Single Node	DPO Multi-node
RM	RM Single Node	RM Multi-node

- Evaluation

- Convert Model Format (Optional)
- Run Evaluation
- Set Up Clusters
- Tips and Tricks
- Citation
- Contributing
- Licenses

Quick Start#

Use this quick start to get going with either the native PyTorch DTensor or Megatron Core training backends.

Note

Both training backends are independent — you can install and use either one on its own.

For more examples and setup details, continue to the Prerequisites section.

Native PyTorch (DTensor)	Megatron Core
Clone and create the environment	

`git clone git@github.com:NVIDIA-NeMo/RL.git nemo-rl --recursive`
`cd nemo-rl`
`uv venv`	

Note: If you previously ran without checking out the submodules, you may need to rebuild virtual	

Native PyTorch (DTensor)	Megatron Core
environments by setting `NRL_FORCE_REBUILD_VENVS=true`. See Tips and Tricks.	
Run GRPO (DTensor) `uv run python examples/run_grpo_math.py`	**Run GRPO (Megatron)** `uv run examples/run_grpo_math.py \ --config examples/configs/grpo_math_1B_megatron.yaml`

Prerequisites#

Clone **NeMo RL**.

git clone git@github.com:NVIDIA-NeMo/RL.git nemo-rl --recursive
cd nemo-rl

If you are already cloned without the recursive option, you can initialize the submodules recursively
git submodule update --init --recursive

Different branches of the repo can have different pinned versions of these third-party submodules. Ensure
submodules are automatically updated after switching branches or pulling updates by configuring git with:
git config submodule.recurse true

NOTE: this setting will not download **new** or remove **old** submodules with the branch's changes.

You will have to run the full `git submodule update --init --recursive` command in these situations.

If you are using the Megatron backend on bare metal (outside of a container), you may need to install the cuDNN headers as well. Here is how you check and install them:

```
# Check if you have libcudnn installed
dpkg -l | grep cudnn.*cuda

# Find the version you need here: https://developer.nvidia.com/cudnn-downloads?target_os=Linux&target_arch=x86_64&Distribution=Ubuntu&target_version=20.04&target_type=deb_network
# As an example, these are the "Linux Ubuntu 20.04 x86_64" instructions
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2004/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
sudo apt install cudnn  # Will install cuDNN meta packages which points to the latest versions
# sudo apt install cudnn9-cuda-12  # Will install cuDNN version 9.x.x compiled for cuda 12.x
# sudo apt install cudnn9-cuda-12-8  # Will install cuDNN version 9.x.x compiled for cuda 12.8
```

If you encounter problems when installing vllm's dependency deep_ep on bare-metal (outside of a container), you may need to install libibverbs-dev as well. Here is how you can install it:

```
sudo apt-get update
sudo apt-get install libibverbs-dev
```

For faster setup and environment isolation, we use uv. Follow these instructions to install uv.

Then, initialize the NeMo RL project virtual environment via:

```
uv venv
```

Note

Please do not use `-p/--python` and instead allow `uv venv` to read it from `.python-version`. This ensures that the version of python used is always what we prescribe.

Use `uv run` to launch all commands. It handles pip installing implicitly and ensures your environment is up to date with our lock file.

Note

- It is not recommended to activate the `venv`, and you should use `uv run <command>` instead to execute scripts within the managed environment. This ensures consistent environment usage across different shells and sessions. Example: `uv run python examples/run_grpo_math.py`
- Ensure your system has the appropriate CUDA drivers installed, and that your PyTorch version is compatible with both your CUDA setup and hardware.
- If you update your environment in `pyproject.toml`, it is necessary to force a rebuild of the virtual environments by setting `NRL_FORCE_REBUILD_VENVS=true` next time you launch a run.
- **Reminder**: Don't forget to set your `HF_HOME`, `WANDB_API_KEY`, and `HF_DATASETS_CACHE` (if needed). You'll need to do a `huggingface-cli login` as well for Llama models.

GRPO#

We provide a reference GRPO configuration for math benchmarks using the OpenInstructMath2 dataset.

You can read about the details of the GRPO implementation here

GRPO Single Node#

To run GRPO on a single GPU for `Qwen/Qwen2.5-1.5B`:

```
# Run the GRPO math example using a 1B parameter model
uv run python examples/run_grpo_math.py
```

By default, this uses the configuration in `examples/configs/grpo_math_1B.yaml`. You can customize parameters with command-line overrides. For example, to run on 8 GPUs,

```
# Run the GRPO math example using a 1B parameter model using 8 GPUs
uv run python examples/run_grpo_math.py \
  cluster.gpus_per_node=8
```

You can override any of the parameters listed in the YAML configuration file. For example,

```
uv run python examples/run_grpo_math.py \
  policy.model_name="meta-llama/Llama-3.2-1B-Instruct" \
  checkpointing.checkpoint_dir="results/llama1b_math" \
  logger.wandb_enabled=True \
  logger.wandb.name="grpo-llama1b_math" \
  logger.num_val_samples_to_print=10
```

The default configuration uses the DTensor training backend. We also provide a config `examples/configs/grpo_math_1B_megatron.yaml` which is set up to use the Megatron backend out of the box.

To train using this config on a single GPU:

```
# Run a GRPO math example on 1 GPU using the Megatron backend
uv run python examples/run_grpo_math.py \
  --config examples/configs/grpo_math_1B_megatron.yaml
```

For additional details on supported backends and how to configure the training backend to suit your setup, refer to the Training Backends documentation.

GRPO Multi-node#

```
# Run from the root of NeMo RL repo
NUM_ACTOR_NODES=2

# grpo_math_8b uses Llama-3.1-8B-Instruct model
COMMAND="uv run ./examples/run_grpo_math.py --config examples/configs/grpo_math_8B.yaml
cluster.num_nodes=2 checkpointing.checkpoint_dir='results/llama8b_2nodes'
logger.wandb_enabled=True logger.wandb.name='grpo-llama8b_math'" \
CONTAINER=YOUR_CONTAINER \
MOUNTS="$PWD:$PWD" \
sbatch \
  --nodes=${NUM_ACTOR_NODES} \
  --account=YOUR_ACCOUNT \
  --job-name=YOUR_JOBNAME \
  --partition=YOUR_PARTITION \
  --time=4:0:0 \
  --gres=gpu:8 \
  ray.sub
```

The required `CONTAINER` can be built by following the instructions in the Docker documentation.

GRPO Qwen2.5-32B#

This section outlines how to run GRPO for Qwen2.5-32B with a 16k sequence length.

```
# Run from the root of NeMo RL repo
NUM_ACTOR_NODES=32

# Download Qwen before the job starts to avoid spending time downloading during the training loop
HF_HOME=/path/to/hf_home huggingface-cli download Qwen/Qwen2.5-32B

# Ensure HF_HOME is included in your MOUNTS
HF_HOME=/path/to/hf_home \
COMMAND="uv run ./examples/run_grpo_math.py --config examples/configs/grpo_math_8B.yaml
policy.model_name='Qwen/Qwen2.5-32B' policy.generation.vllm_cfg.tensor_parallel_size=4
policy.max_total_sequence_length=16384 cluster.num_nodes=${NUM_ACTOR_NODES}
policy.dtensor_cfg.enabled=True policy.dtensor_cfg.tensor_parallel_size=8
policy.dtensor_cfg.sequence_parallel=True policy.dtensor_cfg.activation_checkpointing=True
checkpointing.checkpoint_dir='results/qwen2.5-32b' logger.wandb_enabled=True
logger.wandb.name='qwen2.5-32b'" \
CONTAINER=YOUR_CONTAINER \
MOUNTS="$PWD:$PWD" \
sbatch \
  --nodes=${NUM_ACTOR_NODES} \
  --account=YOUR_ACCOUNT \
  --job-name=YOUR_JOBNAME \
  --partition=YOUR_PARTITION \
  --time=4:0:0 \
  --gres=gpu:8 \
```

ray.sub

GRPO Multi-Turn#

We also support multi-turn generation and training (tool use, games, etc.). Reference example for training to play a Sliding Puzzle Game:

```
uv run python examples/run_grpo_sliding_puzzle.py
```

On-policy Distillation#

We provide an example on-policy distillation experiment using the DeepScaler dataset.

On-policy Distillation Single Node#

To run on-policy distillation on a single GPU using `Qwen/Qwen3-1.7B-Base` as the student and `Qwen/Qwen3-4B` as the teacher:

```
uv run python examples/run_distillation_math.py
```

Customize parameters with command-line overrides. For example:

```
uv run python examples/run_distillation_math.py \
  policy.model_name="Qwen/Qwen3-1.7B-Base" \
  teacher.model_name="Qwen/Qwen3-4B" \
  cluster.gpus_per_node=8
```

On-policy Distillation Multi-node#

```
# Run from the root of NeMo RL repo
NUM_ACTOR_NODES=2

COMMAND="uv run ./examples/run_distillation_math.py --config examples/configs/
distillation_math.yaml cluster.num_nodes=2 cluster.gpus_per_node=8
checkpointing.checkpoint_dir='results/distill_2nodes' logger.wandb_enabled=True
logger.wandb.name='distill-2nodes"' \
CONTAINER=YOUR_CONTAINER \
MOUNTS="$PWD:$PWD" \
sbatch \
  --nodes=${NUM_ACTOR_NODES} \
  --account=YOUR_ACCOUNT \
  --job-name=YOUR_JOBNAME \
  --partition=YOUR_PARTITION \
  --time=4:0:0 \
  --gres=gpu:8 \
  ray.sub
```

Supervised Fine-Tuning (SFT)#

We provide example SFT experiments using various datasets including SQuAD, OpenAI format datasets (with tool calling support), and custom JSONL datasets. For detailed documentation on supported datasets and configurations, see the SFT documentation.

SFT Single Node#

The default SFT configuration is set to run on a single GPU. To start the experiment:

```
uv run python examples/run_sft.py
```

This fine-tunes the `Llama3.2-1B` model on the SQuAD dataset using a 1 GPU.

To use multiple GPUs on a single node, you can modify the cluster configuration. This adjustment will also let you potentially increase the model and batch size:

```
uv run python examples/run_sft.py \
  policy.model_name="meta-llama/Meta-Llama-3-8B" \
  policy.train_global_batch_size=128 \
  sft.val_global_batch_size=128 \
  cluster.gpus_per_node=8
```

Refer to `examples/configs/sft.yaml` for a full list of parameters that can be overridden.

SFT Multi-node#

```
# Run from the root of NeMo RL repo
NUM_ACTOR_NODES=2

COMMAND="uv run ./examples/run_sft.py --config examples/configs/sft.yaml cluster.num_nodes=2
cluster.gpus_per_node=8 checkpointing.checkpoint_dir='results/sft_llama8b_2nodes'
logger.wandb_enabled=True logger.wandb.name='sft-llama8b'" \
CONTAINER=YOUR_CONTAINER \
MOUNTS="$PWD:$PWD" \
sbatch \
    --nodes=${NUM_ACTOR_NODES} \
```

```
  --account=YOUR_ACCOUNT \
  --job-name=YOUR_JOBNAME \
  --partition=YOUR_PARTITION \
  --time=4:0:0 \
  --gres=gpu:8 \
  ray.sub
```

DPO#

We provide a sample DPO experiment that uses the HelpSteer3 dataset for preference-based training.

DPO Single Node#

The default DPO experiment is configured to run on a single GPU. To launch the experiment:

```
uv run python examples/run_dpo.py
```

This trains `Llama3.2-1B-Instruct` on 1 GPU.

If you have access to more GPUs, you can update the experiment accordingly. To run on 8 GPUs, we update the cluster configuration and switch to an 8B Llama3.1 Instruct model:

```
uv run python examples/run_dpo.py \
 policy.model_name="meta-llama/Llama-3.1-8B-Instruct" \
 policy.train_global_batch_size=256 \
 cluster.gpus_per_node=8
```

Any of the DPO parameters can be customized from the command line. For example:

```
uv run python examples/run_dpo.py \
```

```
dpo.sft_loss_weight=0.1 \
dpo.preference_average_log_probs=True \
checkpointing.checkpoint_dir="results/llama_dpo_sft" \
logger.wandb_enabled=True \
logger.wandb.name="llama-dpo-sft"
```

Refer to `examples/configs/dpo.yaml` for a full list of parameters that can be overridden. For an in-depth explanation of how to add your own DPO dataset, refer to the [DPO documentation](#).

DPO Multi-node#

For distributed DPO training across multiple nodes, modify the following script for your use case:

```
# Run from the root of NeMo RL repo
## number of nodes to use for your job
NUM_ACTOR_NODES=2

COMMAND="uv run ./examples/run_dpo.py --config examples/configs/dpo.yaml cluster.num_nodes=2
cluster.gpus_per_node=8 dpo.val_global_batch_size=32 checkpointing.checkpoint_dir='results/
dpo_llama81_2nodes' logger.wandb_enabled=True logger.wandb.name='dpo-llama1b'" \
CONTAINER=YOUR_CONTAINER \
MOUNTS="$PWD:$PWD" \
sbatch \
  --nodes=${NUM_ACTOR_NODES} \
  --account=YOUR_ACCOUNT \
  --job-name=YOUR_JOBNAME \
  --partition=YOUR_PARTITION \
```

```
--time=4:0:0 \
--gres=gpu:8 \
ray.sub
```

RM#

We provide a sample RM experiment that uses the HelpSteer3 dataset for preference-based training.

RM Single Node#

The default RM experiment is configured to run on a single GPU. To launch the experiment:

uv run python examples/run_rm.py

This trains a RM based on `meta-llama/Llama-3.2-1B-Instruct` on 1 GPU.

If you have access to more GPUs, you can update the experiment accordingly. To run on 8 GPUs, we update the cluster configuration:

uv run python examples/run_rm.py cluster.gpus_per_node=8

Refer to the RM documentation for more information.

RM Multi-node#

For distributed RM training across multiple nodes, modify the following script for your use case:

```
# Run from the root of NeMo RL repo
## number of nodes to use for your job
NUM_ACTOR_NODES=2
```

```
COMMAND="uv run ./examples/run_rm.py --config examples/configs/rm.yaml cluster.num_nodes=2
cluster.gpus_per_node=8 checkpointing.checkpoint_dir='results/rm_llama1b_2nodes'
logger.wandb_enabled=True logger.wandb.name='rm-llama1b-2nodes'" \
CONTAINER=YOUR_CONTAINER \
MOUNTS="$PWD:$PWD" \
sbatch \
  --nodes=${NUM_ACTOR_NODES} \
  --account=YOUR_ACCOUNT \
  --job-name=YOUR_JOBNAME \
  --partition=YOUR_PARTITION \
  --time=4:0:0 \
  --gres=gpu:8 \
  ray.sub
```

Evaluation#

We provide evaluation tools to assess model capabilities.

Convert Model Format (Optional)#

If you have trained a model and saved the checkpoint in the PyTorch DCP format, you first need to convert it to the Hugging Face format before running evaluation:

```
# Example for a GRPO checkpoint at step 170
uv run python examples/converters/convert_dcp_to_hf.py \
  --config results/grpo/step_170/config.yaml \
```

```
  --dcp-ckpt-path results/grpo/step_170/policy/weights/ \
  --hf-ckpt-path results/grpo/hf
```

If you have a model saved in Megatron format, you can use the following command to convert it to Hugging Face format prior to running evaluation. This script requires Megatron Core, so make sure you launch with the mcore extra:

```
# Example for a GRPO checkpoint at step 170
uv run --extra mcore python examples/converters/convert_megatron_to_hf.py \
  --config results/grpo/step_170/config.yaml \
  --megatron-ckpt-path results/grpo/step_170/policy/weights/iter_0000000 \
  --hf-ckpt-path results/grpo/hf
```

Note: Adjust the paths according to your training output directory structure.

For an in-depth explanation of checkpointing, refer to the Checkpointing documentation.

Run Evaluation#

Run the evaluation script with the converted model:

```
uv run python examples/run_eval.py generation.model_name=$PWD/results/grpo/hf
```

Run the evaluation script with custom settings:

```
# Example: Evaluation of DeepScaleR-1.5B-Preview on MATH-500 using 8 GPUs
#       Pass@1 accuracy averaged over 16 samples for each problem
uv run python examples/run_eval.py \
  --config examples/configs/evals/math_eval.yaml \
  generation.model_name=agentica-org/DeepScaleR-1.5B-Preview \
```

```
generation.temperature=0.6 \
generation.top_p=0.95 \
generation.vllm_cfg.max_model_len=32768 \
data.dataset_name=math500 \
eval.num_tests_per_prompt=16 \
cluster.gpus_per_node=8
```

> **Note:** Evaluation results may vary slightly due to various factors, such as sampling parameters, random seed, inference engine version, and inference engine settings.

Refer to `examples/configs/evals/eval.yaml` for a full list of parameters that can be overridden. For an in-depth explanation of evaluation, refer to the Evaluation documentation.

Set Up Clusters#

For detailed instructions on how to set up and launch NeMo RL on Slurm or Kubernetes clusters, please refer to the dedicated Cluster Start documentation.

Tips and Tricks#

- If you forget to initialize the NeMo and Megatron submodules when cloning the NeMo-RL repository, you may run into an error like this:

  ```
  ModuleNotFoundError: No module named 'megatron'
  ```

  If you see this error, there is likely an issue with your virtual environments. To fix this, first initialize the submodules:

  ```
  git submodule update --init --recursive
  ```

and then force a rebuild of the virtual environments by setting `NRL_FORCE_REBUILD_VENVS=true` next time you launch a run:

```
NRL_FORCE_REBUILD_VENVS=true uv run examples/run_grpo.py ...
```

- Large amounts of memory fragmentation might occur when running models without support for FlashAttention2. If OOM occurs after a few iterations of training, it may help to tweak the allocator settings to reduce memory fragmentation. To do so, specify `max_split_size_mb` at **either** one of the following places:

1. Launch training with:

```
# This will globally apply to all Ray actors
PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:64 uv run python examples/run_dpo.py ...
```

2. Make the change more permanently by adding this flag in the training configuration:

```
policy:
  # ...
  dtensor_cfg:
    env_vars:
      PYTORCH_CUDA_ALLOC_CONF: "max_split_size_mb:64"
```

Citation#

If you use NeMo RL in your research, please cite it using the following BibTeX entry:

```
@misc{nemo-rl,
title = {NeMo RL: A Scalable and Efficient Post-Training Library},
howpublished = {\url{https://github.com/NVIDIA-NeMo/RL}},
```

year = {2025},
note = {GitHub repository},
}

Contributing#

We welcome contributions to NeMo RL! Please see our Contributing Guidelines for more information on how to get involved.

Licenses#

NVIDIA NeMo RL is licensed under the Apache License 2.0.